

UptonX

RECOVERY USB MANUAL

Emergency recovery procedures, scripts, and cold-boot sequence
for the UptonX homelab infrastructure

KEEP TWO COPIES — ONE OFFSITE

Version 1.1 — March 2026
Generated from uptonx-homelab repository

Table of Contents

Section 1 — Recovery USB Setup

Ventoy Installation

Recommended ISO Layout

Persistence Partition

Keeping ISOs Updated

Section 2 — UptonX Scripts Partition

Directory Structure

Proxmox Scripts

Swarm Scripts

Network Scripts

Frigate Scripts

Nuke Scripts

Section 3 — Boot & Recovery Scenarios

Proxmox Host Won't Boot

Docker Swarm Lost Quorum

CT 106 Destroyed

Traefik Down

Frigate NVR Crashed

NAS Unreachable

Email RAG VM Corrupted

Video AI Stack Down

Audio Pipeline Down

Infisical Down

Home Assistant Down

Network Misconfiguration

Full Power Loss / Cold Boot

Section 4 — Cold Boot Sequence

Complete Startup Order

Verification Checklist

Section 5 — Emergency Credentials

Infrastructure Credentials Card

Section 6 — Physical USB Maintenance

Updating ISOs

Syncing Scripts from Gitea

Labeling and Storage

Section 7 — PBS Backup & Restore

Backup Architecture

Restoring VMs & CTs from PBS

Docker Volume Backup Recovery

Backup Verification

1 Recovery USB Setup

Ventoy multi-boot USB with ISOs and recovery tools

Ventoy Installation

Ventoy creates a multi-boot USB that boots ISO files directly — no need to flash each ISO separately. Drop ISOs onto the USB and Ventoy handles the rest.

Requirements

- USB stick: **64 GB minimum** (128 GB recommended)
- Linux workstation with `ventoy` installed
- All data on the USB will be erased during setup

Installation Steps

- 1 **Download Ventoy** from the official release page. Extract the archive.

```
wget https://github.com/ventoy/Ventoy/releases/download/v1.0.99/ventoy-1.0.99-linux.tar.gz
tar xzf ventoy-1.0.99-linux.tar.gz
cd ventoy-1.0.99
```

- 2 **Identify the USB device** (be absolutely certain — wrong device = data loss).

```
lsblk -o NAME,SIZE,MODEL,TRAN | grep usb
```

- 3 **Install Ventoy** to the USB. This creates two partitions: a large exFAT partition for ISOs and a small EFI partition.

```
# Replace /dev/sdX with your USB device
sudo ./Ventoy2Disk.sh -i -r 8192 /dev/sdX
```

The `-r 8192` flag reserves 8 GB at the end for the UPTONX-RECOVERY partition (created next).

Create the UPTONX-RECOVERY partition in the reserved space.

```
# Create partition in reserved space
sudo parted /dev/sdX mkpart primary fat32 -- -8192MiB -1MiB
# Format as FAT32 with label
sudo mkfs.vfat -F 32 -n UPTONX-RECOVERY /dev/sdX3
```

Tip: Ventoy supports Secure Boot

If you need Secure Boot support, use `sudo ./Ventoy2Disk.sh -i -s /dev/sdX` during installation. The UptonX lab hosts use legacy/UEFI boot without Secure Boot, so this is optional.

Recommended ISO Layout

Copy these ISOs to the Ventoy partition (the large exFAT partition, typically `/dev/sdX1`):

ISO	Purpose	Size
Ubuntu 24.04 LTS (Desktop Live)	Primary recovery OS — SSH, filesystem tools, Docker, networking	~5.7 GB
SystemRescue (latest)	Filesystem repair (fsck, xfs_repair), partition management, data recovery	~900 MB
Clonezilla (latest stable)	Disk imaging — full-disk backup and restore	~400 MB
Proxmox VE Installer (latest 8.x)	Rebuild a Proxmox host from scratch	~1.2 GB
memtest86+ (v7.x)	RAM diagnostics — identify bad memory sticks	~10 MB

Recommended Directory Structure on Ventoy Partition

```
/ventoy/ (Ventoy config – auto-created)
/iso/
├── ubuntu-24.04.2-desktop-amd64.iso
├── systemrescue-11.03-amd64.iso
├── clonezilla-live-3.2.0-5-amd64.iso
├── proxmox-ve_8.3-1.iso
└── memtest86+-7.20.iso
```

Ventoy auto-detects ISOs

Place ISOs anywhere on the Ventoy partition. Using `/iso/` keeps things organized. Ventoy will find and list them all in the boot menu regardless of directory structure.

Persistence Partition Setup

The UPTONX-RECOVERY partition (created in step 4 above) stores recovery scripts, credentials, and documentation that persist across boots. This partition is accessible from any booted ISO as a standard FAT32 volume.

Mount the Recovery Partition After Booting an ISO

```
# Find the UPTONX-RECOVERY partition
lsblk -o NAME,LABEL | grep UPTONX-RECOVERY

# Mount it
sudo mkdir -p /mnt/recovery
sudo mount /dev/sdX3 /mnt/recovery
```

Keeping ISOs Updated

1. Download new ISO versions to your workstation
2. Mount the Ventoy partition: `sudo mount /dev/sdX1 /mnt/ventoy`
3. Replace old ISOs in `/mnt/ventoy/iso/` with new versions
4. Unmount: `sudo umount /mnt/ventoy`
5. No need to re-install Ventoy — it reads ISOs at boot time

Keep Ventoy itself updated

Periodically update Ventoy to support newer ISO formats: `sudo ./Ventoy2Disk.sh -u /dev/sdX` (preserves data).

UptonX Scripts Partition

Recovery scripts organized by subsystem on UPTONX-RECOVERY

Directory Structure

```
UPTONX-RECOVERY (FAT32)
├── scripts/
│   ├── proxmox/
│   │   ├── rejoin-swarm-worker.sh
│   │   ├── restore-lxc-config.sh
│   │   ├── restore-from-pbs.sh
│   │   ├── check-backups.sh
│   │   ├── restore-video-ai.sh
│   │   ├── restore-audio-pipeline.sh
│   │   └── restore-infisical.sh
│   ├── swarm/
│   │   ├── redeploy-all.sh
│   │   └── check-health.sh
│   ├── network/
│   │   ├── remount-nfs.sh
│   │   └── fix-dns.sh
│   ├── frigate/
│   │   ├── check-coral.sh
│   │   └── emergency-restart.sh
│   ├── nuke/
│   │   └── rebuild-ct106.sh
│   └── docs/
│       ├── recovery-usb-manual.pdf
│       └── credentials.pdf
└── isos/ → (symlink to Ventoy ISO directory)
```

These scripts are also committed to the repo

All scripts below live at `docs/scripts/` in the `uptonx-homeLab` repo and are synced to the USB. The canonical version is always in Git.

proxmox/rejoin-swarm-worker.sh

Re-adds a Proxmox host (or workstation) as a Docker Swarm worker after rebuild or reinstall.

Usage: `./rejoin-swarm-worker.sh <worker-ip>`

```
#!/usr/bin/env bash
set -euo pipefail

# Rejoin a Proxmox host as a Docker Swarm worker after rebuild
WORKER_IP="${1:?Usage: $0 <worker-ip>}"
MANAGER="root@192.168.1.23"

echo "====="
echo " Rejoin Swarm Worker: $WORKER_IP"
echo "====="

# Determine SSH user
case "$WORKER_IP" in
    192.168.1.95) WORKER_SSH="chris@$WORKER_IP" ;;
    *)            WORKER_SSH="root@$WORKER_IP" ;;
esac

# Step 1: Verify Docker
echo "[1/5] Checking Docker on worker ..."
ssh "$WORKER_SSH" "docker info --format '{{.ServerVersion}}'" || {
    echo "ERROR: Docker not running. Install: curl -fsSL https://get.docker.com | sh"
    exit 1
}

# Step 2: Leave stale swarm
echo "[2/5] Leaving stale swarm ..."
ssh "$WORKER_SSH" "docker swarm leave --force 2>/dev/null || true"

# Step 3: Get worker join token
echo "[3/5] Getting join token ..."
WORKER_TOKEN=$(ssh "$MANAGER" "docker swarm join-token -q worker")

# Step 4: Join swarm
echo "[4/5] Joining swarm ..."
if [ "$WORKER_IP" = "192.168.1.95" ]; then
    ssh "$WORKER_SSH" "sudo docker swarm join --token $WORKER_TOKEN 192.168.1.23:2377"
else
    ssh "$WORKER_SSH" "docker swarm join --token $WORKER_TOKEN 192.168.1.23:2377"
fi

# Step 5: Apply node labels
echo "[5/5] Applying labels ..."
HOSTNAME=$(ssh "$WORKER_SSH" "hostname")
case "$WORKER_IP" in
    192.168.1.74) ssh "$MANAGER" "docker node update --label-add host=msi $HOSTNAME" ;;
    192.168.1.69) ssh "$MANAGER" "docker node update --label-add host=ai $HOSTNAME" ;;
    192.168.1.95) ssh "$MANAGER" "docker node update --label-add host=workstation --label-add gpu=true $HOSTNAME" ;;
esac

echo "Done! Verify: ssh $MANAGER docker node ls"
```

proxmox/restore-lxc-config.sh

Reapplies the correct AppArmor/nesting configuration to Swarm manager LXC containers (CT 106, 109, 110). Run after any CT config corruption or Proxmox host rebuild.

Usage: `./restore-lxc-config.sh [ctid]` — without args fixes all three managers.

```
#!/usr/bin/env bash
set -euo pipefail

# Required LXC config for Swarm managers:
#   features: keyctl=1,nesting=1
#   lxc.apparmor.profile: unconfined
#   unprivileged: 0

declare -A CT_HOST_MAP=(
    [106]="root@192.168.1.18"   # aux
    [109]="root@192.168.1.77"   # helm
    [110]="root@192.168.1.80"   # aux2
)

fix_ct() {
    local ctid=$1 pve_host=${CT_HOST_MAP[$ctid]}
    echo "Fixing CT $ctid on $pve_host ..."
    ssh "$pve_host" bash -s <<FIXEOF
        CONF="/etc/pve/lxc/${ctid}.conf"
        # Ensure nesting
        grep -q 'nesting=1' "$CONF" || echo 'features: keyctl=1,nesting=1' >> "$CONF"
        # Ensure AppArmor unconfined
        grep -q 'lxc.apparmor.profile: unconfined' "$CONF" || \
            echo 'lxc.apparmor.profile: unconfined' >> "$CONF"
        # Ensure privileged
        sed -i 's/unprivileged: 1/unprivileged: 0/' "$CONF"
        grep -E 'features|apparmor|unprivileged' "$CONF"
    FIXEOF
    echo "Restart CT to apply: ssh $pve_host 'pct stop $ctid && pct start $ctid'"
}

if [ $# -eq 1 ]; then fix_ct "$1"
else for ctid in 106 109 110; do fix_ct "$ctid"; done; fi
```

swarm/redeploy-all.sh

Tears down all Swarm stacks and redeploys them from the repo in the correct order. Run from the repo root or set `REPO_DIR`.

Usage: `./redeploy-all.sh`

```
#!/usr/bin/env bash
set -euv pipefail
MANAGER="root@192.168.1.23"
REPO_DIR="${REPO_DIR:-$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)}"

# 1. Check swarm health
NODE_COUNT=$(ssh "$MANAGER" "docker node ls --format '{{.Status}}' | grep -c Ready")
echo "$NODE_COUNT nodes Ready"

# 2. Remove existing stacks
for stack in $(ssh "$MANAGER" "docker stack ls --format '{{.Name}}'); do
    ssh "$MANAGER" "docker stack rm $stack"
done
sleep 10

# 3. Ensure overlay networks
ssh "$MANAGER" bash -c 'docker network create --driver overlay --attachable proxy 2>/dev/null || true'
ssh "$MANAGER" bash -c 'docker network create --driver overlay --attachable internal 2>/dev/null || true'

# 4. Sync stack files
rsync -az --delete --exclude='.git' "$REPO_DIR/swarm/" "$MANAGER:/opt/stacks/"

# 5. Deploy in order
for stack in portainer whoami landing uptime-kuma grafana cody; do
    ssh "$MANAGER" "docker stack deploy -c /opt/stacks/$stack/stack.yml $stack" 2>/dev/null || true
    sleep 2
done

# 6. Infisical (needs secrets)
ssh "$MANAGER" "test -f /opt/secrets/swarm.env" && \
    ssh "$MANAGER" "set -a && source /opt/secrets/swarm.env && set +a && \
        docker stack deploy -c /opt/stacks/infisical/stack.yml infisical" || \
    echo "SKIP infisical - /opt/secrets/swarm.env not found"

# 7. Verify
ssh "$MANAGER" "docker stack ls && docker service ls"
```

swarm/check-health.sh

Comprehensive health check of all Swarm nodes, services, standalone services, and NFS mounts. Prints a colour-coded status table.

Usage: `./check-health.sh`

```
#!/usr/bin/env bash
set -euv pipefail
MANAGER="root@192.168.1.23"

# Checks: swarm nodes, stack status, service replicas,
#         overlay networks, standalone services (ping),
#         NAS reachability

# Standalone services checked:
#   Traefik (.15:443), Technitium DNS (.51:5380),
#   Frigate (.18:5000), Email RAG (.110:3000),
#   Audio (.74:8090), Home Assistant (.14:8123),
#   CompreFace (.69:8000), Double-Take (.69:3000),
#   Infisical (.69:8093)

# Output: colour-coded table with ✓/✗ status per service
```

network/remount-nfs.sh

Unmounts stale NFS and remounts all NFS shares. Checks NAS reachability first.

Usage: `./remount-nfs.sh [all|email-rag|frigate]`

```
#!/usr/bin/env bash
set -euv pipefail
NAS_IP="192.168.1.11"

# NFS Shares:
#   Email RAG (.110): 192.168.1.11:/volume1/UptonX/email-rag → /mnt/nfs/volumes/email-rag
#   Frigate   (.18):  192.168.1.11:/volume1/UptonX/frigate   → /mnt/nfs/frigate

# Steps:
#   1. Ping NAS to verify reachability
#   2. Check if mount is stale (timeout on ls)
#   3. Force-unmount stale mounts (umount -l)
#   4. Remount with NFSv3, soft, timeo=30
#   5. Verify mount is healthy
```

network/fix-dns.sh

Restores Technitium DNS (CT 101 on aux2, .51) and flushes DNS caches on all hosts.

Usage: `./fix-dns.sh`

```
#!/usr/bin/env bash
set -euo pipefail

# Steps:
# 1. Check CT 101 status on aux2 – start if stopped
# 2. Verify Docker running inside CT
# 3. Check/start Technitium container
# 4. Flush systemd-resolved caches on all Proxmox hosts
# 5. Test DNS resolution for uptonx.com domains
```

frigate/check-coral.sh

Verifies Coral USB TPU detection, VAAPI availability, Frigate container health, camera stream reachability, MQTT status, and NFS archive mount.

Usage: `./check-coral.sh`

```
#!/usr/bin/env bash
set -euo pipefail
FRIGATE_HOST="root@192.168.1.18"

# Checks:
# 1. Coral USB TPU via lsusb (google/coral/global unichip)
# 2. /dev/dri/renderD128 (VAAPI Intel iGPU)
# 3. Frigate container running + API health
# 4. Camera pings on VLAN 40:
#     Reolink Front (40.11), Side (40.12), Back (40.13)
#     Tapo Indoor (40.19), EZVIZ Doorbell (40.50)
# 5. Mosquitto MQTT sidecar
# 6. NFS archive mount (/mnt/nfs/frigate)
```

frigate/emergency-restart.sh

Stops and cleanly restarts Frigate and all sidecars using Docker Compose.

Usage: `./emergency-restart.sh`

Frigate uses docker compose, NEVER docker stack deploy

Frigate requires host networking for camera access on VLAN 40. Docker Swarm mode does not support host networking for services. Always use `docker compose` on aux (.18).

```
#!/usr/bin/env bash
set -euo pipefail
FRIGATE_HOST="root@192.168.1.18"
FRIGATE_DIR="/opt/frigate"

# Steps:
# 1. docker compose down (30s timeout)
# 2. Force-remove orphaned containers
# 3. Verify Coral TPU accessible (lsusb)
# 4. Check/remount NFS archive
# 5. docker compose up -d
# 6. Wait 30s, verify API on :5000
```

nuke/rebuild-ct106.sh

DESTRUCTIVE — Full teardown and rebuild of CT 106

This destroys the primary Swarm manager. ALL Swarm state is lost. Every node (managers and workers) must rejoin the swarm afterward. Only use this when CT 106 is unrecoverable.

```
#!/usr/bin/env bash
set -euo pipefail

# Phases:
# 1. Destroy existing CT 106 on aux (.18)
# 2. Create new CT 106 (2 cores, 2GB RAM, 20GB disk, .23)
#    Config: keyctl=1, nesting=1, apparmor unconfined, privileged
# 3. Install Docker CE
# 4. Configure Docker daemon with TCP API on :2375
#    (Traefik at .15 connects to this for swarm service discovery)
# 5. Configure UFW (SSH, swarm ports, Traefik→Docker API)
# 6. Initialize new Docker Swarm (--advertise-addr 192.168.1.23)
# 7. Create overlay networks: proxy, internal
#
# After running, you must:
# 1. Rejoin Manager 2 (CT 109) and Manager 3 (CT 110)
# 2. Rejoin all workers (.74, .69, .95)
# 3. Recreate /opt/secrets/swarm.env
# 4. Redeploy all stacks
# 5. Restart Traefik on CT 102 (.15)
```

proxmox/restore-from-pbs.sh

Restore a VM or CT from Proxmox Backup Server (.19). Lists available backups, lets you pick a specific date or latest, handles stop/restore/start.

Usage: `./restore-from-pbs.sh <host-ip> <vmid> [snapshot-date]`

```
#!/usr/bin/env bash
set -euo pipefail

# Examples:
# ./restore-from-pbs.sh 192.168.1.69 401          # Email RAG VM (latest)
# ./restore-from-pbs.sh 192.168.1.18 106         # Swarm manager CT (latest)
# ./restore-from-pbs.sh 192.168.1.69 401 2026-03-10 # Specific date
#
# Steps:
# 1. Verify PBS (.19) and Proxmox host are reachable
# 2. List available backups for the VMID
# 3. Select latest or date-specific backup
# 4. Stop running VM/CT if needed (interactive confirm)
# 5. Restore from PBS (qmrestore or pct restore --force)
# 6. Start the restored VM/CT
```

proxmox/check-backups.sh

Verify PBS backups exist and are recent for all VMs/CTs across all Proxmox hosts. Also checks NAS-based Docker volume backups.

Usage: `./check-backups.sh`

```
#!/usr/bin/env bash
set -euo pipefail

# Checks:
# 1. PBS (.19) reachability
# 2. Latest backup per VMID on each Proxmox host
# 3. Age indicator: ✓ (<26h), ▲ (<72h), × (older)
# 4. NAS Docker volume backup directories
#
# Backup schedule reference:
# 02:00 — PBS snapshots (all hosts → .19)
# 03:00 — Docker volume dumps → NAS
# 05:00 — NAS → Google Drive (Sundays)
```

proxmox/restore-video-ai.sh

Restore the Video AI stack (8 containers) on ai NUC (.69). Syncs compose and config files from repo, creates data dirs, starts stack.

Usage: `./restore-video-ai.sh`

All Video AI containers require privileged: true on Proxmox

Docker iptables must also be enabled — daemon.json should be `{}`. The script checks and fixes this automatically.

```
#!/usr/bin/env bash
set -euv pipefail

# Containers restored:
#   compreface-postgres (privileged) – Face recognition DB
#   compreface (:8000→:80) – Exadel CompreFace all-in-one
#   double-take (:3000) – Frigate↔CompreFace bridge
#   qdrant (:6333, :6334) – Vector DB for semantic search
#   n8n (:5678) – Automation workflows
#   clip-indexer – ViT-B-32, polls Frigate, indexes Qdrant
#   llava-bridge – Event narration via Ollama (llava:13b)
#   face-trainer – CompreFace + Telegram ask-me loop
#
# Requires .env for: COMPREFACE_API_KEY, TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID
```

proxmox/restore-audio-pipeline.sh

Restore the Forensic Audio Pipeline on MSI (.74). Two containers: FastAPI backend + React/Vite frontend.

Usage: `./restore-audio-pipeline.sh`

Frontend must be built on the workstation (.95)

Proxmox blocks `child_process.spawn` in Docker build. Use `vm/audio-pipeline/deploy.sh` all from the workstation to build and deploy both images.

```
#!/usr/bin/env bash
set -euv pipefail

# Key config:
#   security_opt: apparmor:unconfined, seccomp:unconfined (NOT privileged)
#   API: 24GB memory limit, 10 CPU cores
#   Whisper: large-v3, CPU int8 quantization
#   DeepFilterNet: v3 neural noise reduction
#   MultipartParser: unlimited file size (was 1MB default)
#   CORS: audio.uptonx.com, localhost:5173, localhost:3000
```

proxmox/restore-infisical.sh

Restore Infisical secrets management on ai NUC (.69). Creates compose file from template if missing, checks .env.

Usage: `./restore-infisical.sh`

```
#!/usr/bin/env bash
set -euo pipefail

# Key config:
#   Standalone Docker Compose (NOT in Swarm – overlay issues)
#   Port: 8093 → 8080 (internal)
#   Postgres: privileged: true (required on Proxmox)
#   Redis: 7-alpine
#
# Required .env variables:
#   INFISICAL_ENCRYPTION_KEY – 32-char hex key
#   INFISICAL_AUTH_SECRET – random secret
#   INFISICAL_POSTGRES_PASSWORD – DB password
```

3 Boot & Recovery Scenarios

Scenario: Proxmox Host Won't Boot (Filesystem Corruption)

SYMPTOMS

- Proxmox host stuck at boot, filesystem errors in console output
- SSH unreachable, web UI at `https://<host-ip>:8006` not loading
- VMs/CTs on that host are down

BOOT

SystemRescue ISO — optimized for filesystem repair and data recovery.

STEPS

1. Boot the affected host from the Recovery USB → select **SystemRescue**
2. Identify the root filesystem:

```
lsblk -f
# Proxmox typically uses: /dev/sda3 (ext4) or LVM on /dev/pve/root
```

3. Run filesystem check:

```
# For ext4:
fsck.ext4 -y /dev/sda3

# For XFS (data drives):
xfs_repair /dev/sdX

# For LVM:
vgscan
vgchange -ay
fsck.ext4 -y /dev/pve/root
```

4. If filesystem is unrecoverable:
 - Backup data: `mount /dev/sdX /mnt/old && rsync -a /mnt/old/etc/pve/ /mnt/recovery/backup/`
 - Reinstall Proxmox using the **Proxmox VE ISO** from the same USB
 - Restore CT/VM configs from the backup or from PBS (.19)
5. Reboot and verify Proxmox web UI loads

VERIFICATION

- Proxmox web UI accessible at `https://<host-ip>:8006`
- All CTs/VMs on the host are running
- If Swarm node: verify with `docker node ls` from manager

Scenario: Docker Swarm Lost Quorum (Majority Managers Down)

SYMPTOMS

- `docker node ls` hangs or returns "Error: rpc error: raft quorum lost"
- Services can't be updated, deployed, or inspected
- At least 2 of 3 managers are down or unreachable

BOOT

No ISO needed — fix from any working machine with SSH access.

STEPS

If only 1 manager is down:

Fix that manager and quorum restores automatically. Only use `force-new-cluster` as a last resort when 2+ managers are down.

1. Find the one surviving manager and force it to become a single-node cluster:

```
# On the surviving manager (e.g., CT 106 at .23):
docker swarm init --force-new-cluster --advertise-addr 192.168.1.23
```

2. Rejoin the other managers:

```
MANAGER_TOKEN=$(docker swarm join-token -q manager)
# From each downed manager CT:
docker swarm leave --force
docker swarm join --token $MANAGER_TOKEN 192.168.1.23:2377
```

3. If no managers survive, rebuild CT 106:

```
./docs/scripts/nuke/rebuild-ct106.sh
```

Then rejoin all other nodes.

VERIFICATION

```
docker node ls
# All 6 nodes should show Status=Ready
# 3 managers, 3 workers
docker service ls
# All services should show correct replica counts
```

Scenario: CT 106 (Primary Swarm Manager) Destroyed

SYMPTOMS

- CT 106 on aux (.18) is destroyed, corrupted, or unrecoverable
- Traefik can't reach Docker API at tcp://192.168.1.23:2375
- All *.uptonx.com Swarm services are unreachable (Traefik has no backend)

BOOT

No ISO needed — run rebuild script from workstation or another host with SSH access to aux (.18).

STEPS

1. Run the nuke/rebuild script:

```
./docs/scripts/nuke/rebuild-ct106.sh
```

2. Script will output join tokens. Rejoin all other nodes using the tokens printed.
3. Redeploy stacks: `./docs/scripts/swarm/redeploy-all.sh`
4. Restart Traefik to reconnect to new Docker API:

```
ssh root@192.168.1.15 "cd /opt/traefik && docker compose restart"
```

VERIFICATION

- `docker node ls` shows all 6 nodes
- `docker service ls` shows all services with correct replicas
- `curl -s https://whoami.uptonx.com` returns a response

Scenario: Traefik Down (All *.uptonx.com Unreachable)

SYMPTOMS

- All *.uptonx.com domains return connection refused or timeout
- Direct IP access to services still works (e.g., http://192.168.1.18:5000)

BOOT

No ISO needed — SSH into CT 102 on aux2 (.80).

STEPS

1. Check CT 102 is running:

```
ssh root@192.168.1.80 "pct status 102"
# If stopped:
ssh root@192.168.1.80 "pct start 102"
```

2. SSH into CT 102 and check Traefik:

```
ssh root@192.168.1.15 "cd /opt/traefik && docker compose ps"
# If not running:
ssh root@192.168.1.15 "cd /opt/traefik && docker compose up -d"
```

3. Check Traefik can reach the Docker API:

```
ssh root@192.168.1.15 "curl -s http://192.168.1.23:2375/version"
# If this fails, check CT 106 Docker daemon
```

4. Check TLS certificates:

```
ssh root@192.168.1.15 "ls -la /opt/traefik/acme.json"
# If missing/empty, Traefik will auto-request via Cloudflare DNS challenge
```

VERIFICATION

```
curl -s -o /dev/null -w "%{http_code}" https://whoami.uptonx.com
# Should return 200
```

Scenario: Frigate NVR Crashed (Cameras Offline)

SYMPTOMS

- Camera feeds not loading in Frigate UI or Home Assistant
- `http://192.168.1.18:5000` not responding
- No new recordings appearing

BOOT

No ISO needed — SSH into aux (.18).

STEPS

1. Run the hardware check first:

```
./docs/scripts/frigate/check-coral.sh
```

2. If containers are down, run emergency restart:

```
./docs/scripts/frigate/emergency-restart.sh
```

3. If Coral TPU is not detected:

- Physically reseal the Coral USB device
- Check `dmesg | grep apex` for driver errors
- Frigate will fall back to CPU detection (slower but functional)

4. If NFS archive is stale: `./docs/scripts/network/remount-nfs.sh frigate`

VERIFICATION

- Frigate UI loads at `http://192.168.1.18:5000`
- All 5 cameras show live feeds
- Frigate API: `curl -s http://192.168.1.18:5000/api/stats`

Scenario: NAS Unreachable (NFS Mounts Stale Everywhere)

SYMPTOMS

- Commands that touch NFS paths hang (ls, df, du)
- Email RAG VM containers failing to start
- Frigate NAS archival failing
- `ping 192.168.1.11` fails

BOOT

No ISO needed — physical access to NAS required.

STEPS

1. **Check physical NAS:** Power button, network cable, status LEDs on the UGreen NAS
2. **If NAS is off:** Power it on and wait 3-5 minutes for NFSv3 service to start
3. **If NAS is on but unreachable:** Check the network switch and cable connections
4. Once NAS is reachable, remount all NFS shares:

```
./docs/scripts/network/remount-nfs.sh all
```

5. Restart affected services:

```
# Email RAG
ssh chris@192.168.1.110 "cd /opt/email-rag && docker compose restart"

# Frigate (if archival was failing)
ssh root@192.168.1.18 "cd /opt/frigate && docker compose restart"
```

VERIFICATION

```
ping -c 3 192.168.1.11
ssh chris@192.168.1.110 "ls /mnt/nfs/volumes/email-rag"
ssh root@192.168.1.18 "ls /mnt/nfs/frigate"
```

Scenario: Email RAG VM Corrupted

SYMPTOMS

- `email-rag.uptonx.com` or `chat.uptonx.com` unreachable
- VM 401 on ai NUC (.69) won't start or SSH fails to .110

BOOT

No ISO needed — access ai NUC Proxmox UI or SSH.

STEPS

1. Check VM status from Proxmox:

```
ssh root@192.168.1.69 "qm status 401"
# If stopped:
ssh root@192.168.1.69 "qm start 401"
```

2. If VM boots but services are down:

```
ssh chris@192.168.1.110 "cd /opt/email-rag && docker compose up -d"
```

3. If NFS is stale:

```
./docs/scripts/network/remount-nfs.sh email-rag
```

4. If VM is completely unrecoverable:

- Restore from PBS backup: `ssh root@192.168.1.69 "qmrestore /path/to/backup 401"`
- Or rebuild: Create new Ubuntu 24.04 VM, follow `vm/email-rag/` setup in repo

VERIFICATION

```
curl -s http://192.168.1.110:3000/health
curl -s http://192.168.1.110:8080 # Open WebUI
```

Scenario: Video AI Stack Down (CompreFace / Double-Take / Qdrant)

SYMPTOMS

- `faces.uptonx.com` , `dt.uptonx.com` , or `n8n.uptonx.com` unreachable
- Face recognition not working in Frigate/Double-Take
- Semantic search broken (Qdrant down)

BOOT

No ISO needed — SSH into ai NUC (.69).

STEPS

1. Check Docker status on ai NUC:

```
ssh root@192.168.1.69 "docker ps --format 'table {{.Names}}\t{{.Status}}'"
```

2. If containers are down, run the restore script:

```
./docs/scripts/proxmox/restore-video-ai.sh
```

3. If Docker iptables is disabled (containers can't reach each other):

```
ssh root@192.168.1.69 "echo '{}' > /etc/docker/daemon.json && systemctl restart docker"
```

4. If `face_trainer` or `clip_indexer` fails, check `.env`:

```
ssh root@192.168.1.69 "cat /opt/video-ai/.env"
# Needs: COMPREFACE_API_KEY, TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID
```

VERIFICATION

```
curl -s http://192.168.1.69:8000 # CompreFace
curl -s http://192.168.1.69:3000 # Double-Take
curl -s http://192.168.1.69:6333 # Qdrant
curl -s http://192.168.1.69:5678 # n8n
```

Scenario: Audio Pipeline Down

SYMPTOMS

- `audio.uptonx.com` unreachable or returns 502
- Transcription/analysis not working

BOOT

No ISO needed — SSH into MSI (.74).

STEPS

1. Check container status:

```
ssh root@192.168.1.74 "cd /opt/stacks/audio-pipeline && docker compose ps"
```

2. If containers are down:

```
./docs/scripts/proxmox/restore-audio-pipeline.sh
```

3. If images are missing (after Docker prune or reinstall):

```
# Must rebuild from workstation:  
cd ~/uptonx-homelab/vm/audio-pipeline  
./deploy.sh all
```

4. Check API logs for model loading issues:

```
ssh root@192.168.1.74 "cd /opt/stacks/audio-pipeline && docker compose logs --  
tail 50 audio_api"
```

VERIFICATION

```
curl -s http://192.168.1.74:8000/docs # API docs  
curl -s http://192.168.1.74:8090 # Frontend
```

Scenario: Infisical Down (Secrets Unavailable)

SYMPTOMS

- `infisical.uptonx.com` unreachable
- Services that depend on secrets can't start

BOOT

No ISO needed — SSH into ai NUC (.69).

STEPS

1. Check Infisical containers:

```
ssh root@192.168.1.69 "cd /opt/infisical && docker compose ps"
```

2. Run restore script:

```
./docs/scripts/proxmox/restore-infisical.sh
```

3. If `.env` is missing, generate new secrets:

```
ssh root@192.168.1.69 bash -c 'cat > /opt/infisical/.env <<EOF
INFISICAL_ENCRYPTION_KEY=$(openssl rand -hex 16)
INFISICAL_AUTH_SECRET=$(openssl rand -base64 32)
INFISICAL_POSTGRES_PASSWORD=$(openssl rand -base64 16)
EOF'
```

4. If postgres data is corrupted, restore from PBS:

```
./docs/scripts/proxmox/check-backups.sh # Find latest backup
# Restore infisical-postgres volume from NAS backup
```

VERIFICATION

```
curl -s -o /dev/null -w "%{http_code}" http://192.168.1.69:8093
# Should return 200 or 302
```

Scenario: Home Assistant Down

SYMPTOMS

- `http://192.168.1.14:8123` unreachable
- Tablet dashboard blank, automations not running
- Voice pipeline (Whisper/Piper) unresponsive

BOOT

No ISO needed — HA OS runs on its own appliance at .14.

STEPS

1. Check if HA is reachable:

```
ping 192.168.1.14
```

If unreachable, check physical device (power, network cable).

2. If HA OS is reachable but web UI is down, SSH in:

```
ssh root@192.168.1.14 # Password: Terry87!
ha core start          # Start HA Core
ha core info           # Check status
ha supervisor info     # Check supervisor
```

3. Check addon status:

```
ha addons              # List all addons
ha addons start core_ssh # Restart SSH addon if needed
```

4. If HA needs full restore from backup:

```
# From HA CLI:
ha backups          # List available backups
ha backups restore <slug> # Restore specific backup

# Or from web UI (if accessible):
# Settings → System → Backups → select backup → Restore
```

5. If voice pipeline is down:

```
ha addons restart core_whisper # Whisper STT (:10300)
ha addons restart core_piper   # Piper TTS (:10200)
```

HA Backup Best Practice

HA should be configured to create automatic backups. Check Settings → System → Backups.

Consider copying backups to NAS for offsite recovery: `scp root@192.168.1.14:/backup/
*.tar /mnt/nfs/backups/ha/`

VERIFICATION

- Web UI loads at `http://192.168.1.14:8123`
- Tablet (.91) reconnects to dashboard
- Frigate integration shows camera feeds
- Automations running (Night Dim, Morning Restore)

Scenario: Network Misconfiguration (Can't Reach Anything)

SYMPTOMS

- Multiple hosts unreachable, DNS not resolving
- Gateway (.1) may or may not be reachable

BOOT

Ubuntu 24.04 Live ISO — boot on the workstation for a clean network stack.

STEPS

1. Boot Ubuntu Live from USB on the workstation
2. Check basic connectivity:

```
# Can we reach the gateway?
ping 192.168.1.1

# Can we reach any Proxmox host?
ping 192.168.1.18 # aux
ping 192.168.1.77 # helm
ping 192.168.1.80 # aux2
```

3. If gateway is unreachable:
 - Check physical cable from workstation to switch
 - Check UniFi Cloud Gateway (.1) — power cycle if needed
 - Login: `admin / Terry1987!!!`
4. If gateway works but hosts are down:
 - Check TP-Link switch (.13) — VLAN config may be wrong
 - Verify Proxmox hosts are powered on
5. If DNS is the issue:

```
./docs/scripts/network/fix-dns.sh
```

6. Verify DNS config:

```
# All hosts should use Technitium (.51) as DNS
# If systemd-resolved is pointing elsewhere, fix:
sudo resolvectl dns eth0 192.168.1.51
```

VERIFICATION

```
ping 192.168.1.1    # gateway
ping 192.168.1.51   # DNS
dig @192.168.1.51 uptonx.com # should resolve to .15
```

Scenario: Full Homelab Power Loss / Cold Boot

SYMPTOMS

- Everything is off — complete power failure or intentional shutdown
- Nothing is reachable on the network

BOOT

No ISO needed — follow the Cold Boot Sequence in Section 4.

STEPS

See **Section 4 — Cold Boot Sequence** for the complete ordered startup procedure.

4

Cold Boot Sequence

Correct order to bring up UptonX from completely powered off

Order matters!

Services have dependencies. Starting things out of order will cause cascading failures. Follow this sequence exactly.

Complete Startup Order

Step	Action	Wait For	Verify
1	Power on NAS (192.168.1.11)	3-5 min for NFS shares to be available	<code>ping 192.168.1.11</code>
2	Power on UniFi Gateway (.1) and switches	2-3 min for DHCP and routing	<code>ping 192.168.1.1</code>
3	Power on Proxmox: aux2 (.80) first	2-3 min for Proxmox to boot	<code>https://192.168.1.80:8006</code>
4	Power on Proxmox: aux (.18), helm (.77)	2-3 min each	Web UI on each host
5	Power on Proxmox: ai (.69), msi (.74)	2-3 min each	Web UI / SSH
6	Start CT 101 (Technitium DNS) on aux2	30 sec	<code>dig @192.168.1.51 uptonx.com</code>
7	Start CT 102 (Traefik) on aux2	30 sec	<code>curl -s https://uptonx.com</code>
8	Start Swarm managers: CT 106 (aux), CT 109 (helm), CT 110 (aux2)	1-2 min for Raft consensus	<code>ssh root@192.168.1.23 "docker node ls"</code>
9	Verify Swarm quorum (3 managers Ready)	—	All 3 managers show "Leader" or "Reachable"
10	Verify workers (.74, .69, .95) are Ready	—	<code>docker node ls</code> — all 6 nodes Ready
11	Check Swarm stacks are running	—	<code>docker service ls</code> — correct replicas
12	If stacks need redeployment: <code>./swarm/deploy-all.sh</code>	1-2 min	<code>docker stack ls</code>
13	Start Frigate on aux (.18)	30-60 sec	<code>curl http://192.168.1.18:5000/api/version</code>
14	Start Video AI stack on ai NUC (.69)	1-2 min	<code>curl http://192.168.1.69:8000</code>
15	Start Email RAG VM 401 on ai NUC	1-2 min	<code>curl http://192.168.1.110:3000</code>
16	Remount NFS shares if stale	—	<code>./docs/scripts/network/remount-nfs.sh all</code>

17	Verify all *.uptonx.com URLs	—	See URL checklist below
18	Verify Home Assistant (.14)	—	http://192.168.1.14:8123
19	Check HA tablet (.91) reconnects	—	Tablet shows dashboard

URL Verification Checklist

URL	Service	Expected
https://uptonx.com	Landing page	200 OK
https://whoami.uptonx.com	Whoami (test)	200 OK
https://portainer.uptonx.com	Portainer	Login page
https://status.uptonx.com	Uptime Kuma	Status page
https://grafana.uptonx.com	Grafana	Login page
https://frigate.uptonx.com	Frigate NVR	Camera dashboard
https://email-rag.uptonx.com	Email RAG API	200 OK
https://chat.uptonx.com	Open WebUI	Chat interface
https://audio.uptonx.com	Audio Pipeline	200 OK
https://dt.uptonx.com	Double-Take	Dashboard
https://faces.uptonx.com	CompreFace	API response
https://n8n.uptonx.com	n8n	Login page
https://infisical.uptonx.com	Infisical	Login page

Quick verification one-liner

```
for url in uptonx.com whoami.uptonx.com portainer.uptonx.com \
status.uptonx.com grafana.uptonx.com frigate.uptonx.com \
email-rag.uptonx.com chat.uptonx.com audio.uptonx.com \
dt.uptonx.com faces.uptonx.com n8n.uptonx.com infisical.uptonx.com; do
code=$(curl -s -o /dev/null -w "%{http_code}" --connect-timeout 5 "https://$url")
printf "%-30s %s\n" "$url" "$code"
done
```

Emergency Credentials

Print-ready credentials card — keep with the USB stick

SENSITIVE — Handle with care

This page contains all infrastructure credentials. Keep the printed copy with the USB stick in a secure location. Destroy old copies when credentials change.

Proxmox Hosts

Host	IP	User	Password	Web UI
helm	192.168.1.77	root	Terry87!	:8006
aux	192.168.1.18	root	Terry87!	:8006
aux2	192.168.1.80	root	Terry87!	:8006
ai (NUC)	192.168.1.69	root	Terry87!	:8006
msi	192.168.1.74	root	Terry87!	:8006
PBS	192.168.1.19	root	Terry87!	:8007

LXC Containers

Container	IP	Root Password
CT 101 (Technitium DNS)	192.168.1.51	Terry87!
CT 102 (Traefik)	192.168.1.15	Terry87!
CT 106 (Swarm Mgr 1)	192.168.1.23	Terry87!
CT 109 (Swarm Mgr 2)	192.168.1.26	Terry87!
CT 110 (Swarm Mgr 3)	192.168.1.27	Terry87!

VMs & Services

Service	IP / URL	User	Password
Email RAG VM	192.168.1.110	chris	Zooey87!
Home Assistant	192.168.1.14:8123	chris	Zooey87!
HA SSH	192.168.1.14:22	root	Terry87!
Fully Kiosk	192.168.1.91:2323	—	Terry87! (PIN: 8787)
NAS	192.168.1.11	chris-admin	—

Network Infrastructure

Device	IP	User	Password
UniFi Cloud Gateway	192.168.1.1	admin	Terry1987!!!
TP-Link Switch	192.168.1.13	admin	(default)
Technitium DNS UI	192.168.1.51:5380	admin	Terry87!

Cameras (VLAN 40)

Camera	IP	User	Password
Reolink Front	192.168.40.11	admin	Terry!87
Reolink Side	192.168.40.12	admin	Terry!87
Reolink Back	192.168.40.13	admin	Terry!87
Tapo Indoor	192.168.40.19	chris87	Terry!87
EZVIZ Doorbell	192.168.40.50	—	(app-managed)

External / Git

Service	URL	Notes
Gitea	http://192.168.1.58:3000	Homelab repo source of truth
Cloudflare	dash.cloudflare.com	DNS + tunnel for uptonx.com
Infisical	https://infisical.uptonx.com	Secrets management

Physical USB Maintenance

Keeping the recovery USB current and accessible

How to Update ISOs on Ventoy

1. Plug in the recovery USB on the workstation
2. Mount the Ventoy partition:

```
sudo mount /dev/sdX1 /mnt/ventoy
```

3. Replace old ISOs with new versions in `/mnt/ventoy/iso/`
4. Update Ventoy bootloader if needed:

```
cd ~/ventoy-*  
sudo ./Ventoy2Disk.sh -u /dev/sdX # preserves data
```

5. Unmount: `sudo umount /mnt/ventoy`

How to Sync Latest Scripts from Gitea

```
# Mount the UPTONX-RECOVERY partition  
sudo mount /dev/sdX3 /mnt/recovery  
  
# Clone or pull latest from Gitea  
cd /tmp  
git clone http://192.168.1.58:3000/chris/uptonx-homelab.git  
# Or if already cloned:  
cd uptonx-homelab && git pull  
  
# Sync scripts to USB  
rsync -av --delete docs/scripts/ /mnt/recovery/scripts/  
  
# Copy latest PDF  
cp docs/recovery-usb-manual.pdf /mnt/recovery/docs/  
  
# Unmount  
sudo umount /mnt/recovery
```

Automated sync script

Create a simple script on the workstation to automate this:

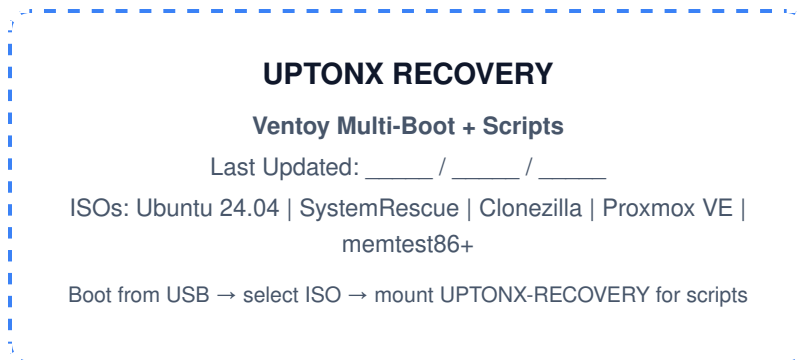
```
#!/bin/bash
# sync-recovery-usb.sh
USB_PART=$(lsblk -o NAME,LABEL -nr | grep UPTONX-RECOVERY | awk '{print $1}')
if [ -z "$USB_PART" ]; then echo "USB not found!"; exit 1; fi
sudo mount "/dev/$USB_PART" /mnt/recovery
cd ~/uptonx-homelab && git pull
rsync -av --delete docs/scripts/ /mnt/recovery/scripts/
cp docs/recovery-usb-manual.pdf /mnt/recovery/docs/
sudo umount /mnt/recovery
echo "Sync complete!"
```

Recommended: Keep Two USB Sticks

- **USB #1:** Keep with the homelab rack — immediate access during emergencies
- **USB #2:** Keep offsite (office, safe deposit box) — disaster recovery
- Update both whenever you sync scripts or update ISOs
- Both should contain identical ISOs and scripts

Label and Date the USB Stick

Use a label maker or permanent marker. Include:



Quarterly Maintenance Checklist

Task	Frequency
Check for new ISO versions and update	Quarterly
Sync scripts from Gitea	After any infra change
Update Ventoy bootloader	When new version available
Test-boot on a spare machine	Quarterly
Verify scripts still work (dry run)	Quarterly
Update credentials if changed	Immediately
Sync offsite USB copy	After any update
Update "Last Updated" date on label	After any update
Run <code>check-backups.sh</code> to verify PBS	Weekly

PBS Backup & Restore

Proxmox Backup Server architecture, restore procedures, and verification

Backup Architecture

UptonX uses a three-tier backup strategy with automated schedules:

Time	What	Where	Retention
02:00 daily	PBS snapshots — all VMs & CTs on every Proxmox host	PBS at 192.168.1.19	14 days
03:00 daily	Docker volume dumps — PostgreSQL, SQLite, configs	NAS at 192.168.1.11	14 days
05:00 Sundays	Off-site sync — NAS → Google Drive	Google Drive (rclone)	Cloud-managed

Docker Volume Backup Coverage

Host	What Gets Backed Up	How
ai NUC (.69)	compreface-postgres, infisical-postgres, qdrant vectors, double-take config, n8n data	pg_dump + volume tar
aux (.18)	frigate config, frigate.db	DB copy from /media/frigate/
aux2 (.80)	traefik ACME certs, authentik-postgresql, authentik-media	pg_dump + cert copy
msi (.74)	audio-pipeline SQLite (jobs.db)	File copy
email-rag (.110)	PostgreSQL (email_rag database)	pg_dump via sudo
Swarm Mgr (.23)	grafana_data, prometheus, loki_data, portainer_data	Docker volume tar

NAS archive path: `192.168.1.11:/volume1/UptonX/backups/{hostname}/`

Restoring VMs & CTs from PBS

Method 1: Using the restore script

```
# Restore latest backup of Email RAG VM
./docs/scripts/proxmox/restore-from-pbs.sh 192.168.1.69 401

# Restore specific date
./docs/scripts/proxmox/restore-from-pbs.sh 192.168.1.69 401 2026-03-10

# Restore CT 106 (Swarm Manager)
./docs/scripts/proxmox/restore-from-pbs.sh 192.168.1.18 106
```

Method 2: From Proxmox Web UI

1. Open Proxmox web UI: `https://<host-ip>:8006`
2. Navigate to **Datacenter** → **Storage** → **pbs**
3. Select the backup you want to restore
4. Click **Restore**, select target VMID and storage
5. Check "Start after restore" if desired

Method 3: CLI (manual)

```
# List available backups
pvesh get /nodes/$(hostname)/storage/pbs/content --vmid 401

# Restore a VM
qmrestore <backup-volid> 401 --force

# Restore a CT
pct restore 106 <backup-volid> --force

# Start after restore
qm start 401 # or: pct start 106
```

Docker Volume Backup Recovery

Docker volume backups are stored on the NAS. To restore:

PostgreSQL databases

```
# Find latest dump on NAS
ssh root@192.168.1.69 "ls -lt /mnt/nfs/backups/ai/*.sql.gz | head -5"

# Restore compreface postgres
ssh root@192.168.1.69 "gunzip -c /mnt/nfs/backups/ai/compreface-latest.sql.gz | \
    docker exec -i compreface-postgres psql -U compreface"

# Restore infisical postgres
ssh root@192.168.1.69 "gunzip -c /mnt/nfs/backups/ai/infisical-latest.sql.gz | \
    docker exec -i infisical-postgres psql -U infisical"

# Restore email-rag postgres (native, not Docker)
ssh chris@192.168.1.110 "gunzip -c /mnt/nfs/backups/email-rag/email_rag-latest.sql.gz | \
    sudo -u postgres psql email_rag"
```

Docker volumes (tar archives)

```
# Restore Grafana data
ssh root@192.168.1.23 "docker volume create grafana_data && \
    docker run --rm -v grafana_data:/data -v /mnt/nfs/backups/swarm:/backup alpine \
    tar xzf /backup/grafana_data-latest.tar.gz -C /data"
```

Frigate database

```
# Stop Frigate first
ssh root@192.168.1.18 "cd /opt/frigate && docker compose stop frigate"

# Restore frigate.db
ssh root@192.168.1.18 "cp /mnt/nfs/backups/aux/frigate.db /media/frigate/frigate.db"

# Restart
ssh root@192.168.1.18 "cd /opt/frigate && docker compose start frigate"
```

Backup Verification

Run the backup check script to verify all backups are current:

```
./docs/scripts/proxmox/check-backups.sh
```

This checks:

- PBS server (.19) reachability
- Latest backup age per VMID on each Proxmox host
- NAS Docker volume backup directories

Backup age indicators

- ✓
 - backup < 26 hours old (healthy)
- ⚠
 - backup 26-72 hours old (investigate)
- ✗
 - backup > 72 hours old (action needed)

Manual PBS verification

```
# PBS Web UI
https://192.168.1.19:8007

# List all backups from CLI (on any Proxmox host)
pvesh get /nodes/$(hostname)/storage/pbs/content

# Check PBS datastore status
ssh root@192.168.1.19 "proxmox-backup-manager datastore list"
```

Off-site backup verification

Check Google Drive sync status: `rcclone ls gdrive:backups/ | head -20` (run from workstation where rclone is configured).